

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 3 — Data Types and Casting

Five types, four conversions, and the trap you already met — OCR J277 §2.2.2

Where this came from

In **Programming 1** you wrote a program that asked for a name, asked for a number, and greeted someone that many times. It only worked because of **one** piece of code: `int()`.

Programming 2 was about that program's *shape* — sequence, then iteration. **Today is about what is actually inside it.** Everything a program does is done to a value of some type, and picking the wrong type is one of the commonest ways a working-looking program goes wrong.

The five data types — and OCR wants all five by name

OCR name	What it holds	Python name	Example
Integer	A whole number. No decimal point.	<code>int</code>	7
Real	A number <i>with</i> a decimal point.	<code>float</code>	9.81
Boolean	One of exactly two values: <code>True</code> / <code>False</code> .	<code>bool</code>	<code>True</code>
Character	One single character.	<code>str</code>	<code>"A"</code>
String	Text of any length. Can contain digits and punctuation.	<code>str</code>	<code>"Batman"</code>

VOCAB Data type (OCR §2.2.2) — what *kind* of value a variable holds — which decides what the computer will let you do with it

Python has no separate Character type — `"A"` is a String one character long. **Learn all five names anyway: the exam asks for them.**

Ask Python what type something is

```
print(type(7))
print(type(9.81))
print(type("Batman"))
print(type(True))
```

Now try • Run it. Write down the four words it prints.

Now try • Now try `type("7")` — with the quote marks. *Why is that different from `type(7)`?*

Now try • Try `type(input("Anything: "))`. Type a number when it asks. What does it say, and does that surprise you?

Choosing the right type — this is the exam skill

§2.2.2 asks you to **choose a suitable data type for a scenario**. That is a real exam question, and it is worth marks. Fill this in.

What you need to store	Data type	Why?
The number of students in a class		
The price of a train ticket		
Whether a user is logged in		
A pupil's exam grade, as one letter		
A UK mobile phone number		

The phone number is a trap, and it is on real papers

It *looks* like a number. It is not a number — it is a **String**. Two reasons, and you need both:

1. The leading zero disappears. Try it:

```
print(int("01334462000"))
```

Python prints 1334462000. The 0 is gone, because numbers do not have leading zeros — and now it is not a phone number any more.

2. You never do arithmetic on it. You never add two phone numbers together, or find the average of a class's phone numbers. **If you will never do sums with it, it is probably a String** — and that rule also covers postcodes, bank account numbers and student ID numbers.

One warning about the online **Data Types Quiz**: it lists **Date & Time** as a data type. **OCR does not**. Stick to the five in the table above.

Casting — changing a value from one type to another

Casting is converting a value from one data type into another. Four functions do almost all of it.

Function	Turns something into	Example
<code>int()</code>	a whole number	<code>int("42")</code> gives 42
<code>float()</code>	a decimal number	<code>float("4.52")</code> gives 4.52
<code>str()</code>	text	<code>str(345)</code> gives "345"
<code>bool()</code>	True or False	<code>bool(1)</code> gives True

VOCAB Casting (OCR §2.2.2) — temporarily changing a value from one data type to another — and where that is useful

Four traps. Run every one of them.

Each of these is a real error you will hit this year. Cause it *now*, on purpose, while nothing depends on it.

Trap 1 — `input()` always gives you a String. Always. Even when the user types a number. This is the whole reason `int()` exists.

Trap 2 — `int()` cannot swallow a decimal.

```
print(int("3.5"))
```

You might expect 3. You get `ValueError`. If you need decimals, the tool is `float()`, not `int()`.

Trap 3 — you cannot glue a number onto text with `+`.

```
strength = 8
print("My strength is " + strength)
```

`TypeError`. The fix is `str(strength)` — cast the number into text, then the `+` works. **Use this one.** It is also exactly how the exam writes it.

Trap 4 — the one that does not warn you.

```
print("3" * 4)
```

No error. No red text. It prints 3333, because `*` on a String means *repeat it*. **That is a logic error** — the program ran perfectly and gave the wrong answer. They are the hardest kind to find, because nothing tells you.

Now try • Predict what `bool("False")` gives, then run it. *Were you right?*

Now try • Try `bool("")` — empty quote marks. And `bool(0)`.

Now try • From those three results, work out the rule Python is actually using.

Challenge 1 — My Superhero

Go to 101computing.net/my-superhero. Fill in the starter variables, then print a full description, like:

“My name is Batman and I live in Gotham City. I am male and my enemy is The Joker.”

The point of this challenge is Trap 3. speed, strength and age are numbers; the rest are text. The moment you try to print them in one sentence, you will need `str()`.

Now try • Get the description printing on one line.

Now try • Add a sentence using strength — an Integer — inside it.

Now try • Now add a Real. Give your superhero a height in metres — 1.88, not 188. Put it in the description.

Now try • Now make the computer produce one. Add an agility score, then print the *average* of speed, strength and agility.

That last one has a surprise in it. Even if your three scores are whole numbers that divide exactly, Python prints 8.0 — not 8. **Dividing always gives you a Real**, whatever you divide. Check it with `type()` if you do not believe it.

Challenge 2 — Weight on the Moon

101computing.net/weight-on-the-moon-calculator

Ask the user their weight, then print what they would weigh on the Moon (multiply by 0.165). **Which cast do you need here — `int()` or `float()`?** Decide before you type, then find out whether you were right.

Challenge 3 — The Character Builder

The long one. *My Superhero* used values you typed straight into the code. This time the *user* supplies everything — so every value arrives as a `String` and **you** decide what it should become.

Write a program that builds a character sheet. It must use **all five data types**.

Ask the user for	Example	Data type	Which cast?
Character name	Ripley	String	<i>none — it is already text</i>
Level	7	Integer	
Health remaining	87.5	Real	
Rank, as one letter	A	Character	
Strength (out of 10)	8	Integer	
Speed (out of 10)	6	Integer	

Then print a character sheet, using `str()` wherever you need it:

```
--- CHARACTER SHEET ---
Name:  Ripley  (rank A)
Level:  7
Health: 87.5
Power:  7.0
```

Power is the interesting one. It is the average of strength and speed — so *you* do not type a `Real` anywhere, and one comes out anyway. Two `Integers` went in. **Say why in one sentence** before you move on:

Now try • Get the five inputs working first, with the right cast on each one.

Now try • Print the sheet exactly as laid out above.

Now try • Ask for a Level of 3.5 and read the error. *Which cast rejected it, and why is that the correct behaviour?*

Now try • **Stretch — the fifth type.** Ask “Still alive? (yes/no)” and store the answer as a real **Boolean**, not as the text “yes”. One line does it: `alive = (answer == "yes")`. *Check it with `type()`.*

Now write it out — by hand

Copy your finished **Character Builder** out here by hand — the whole program, not a summary. **Paper 2 is written on paper:** no colours, no autocomplete, no Run button. Watch your **indents**, your **brackets** (especially the double ones in `int(input(...))`), and **which cast is on each line**.

main.py

1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	
20	
21	
22	
23	
24	
25	
26	
27	
28	

Save your work before you close the tab — Download, or paste it into your Drive. The editor only remembers it in this browser, on this device.